

1. The Essence of my Solution to the Challenge

My Postfix++ calculator solution is built on a combination of the fundamental data structures and algorithmic methods that make up this module's core. To evaluate postfix expressions, the stack is the main data structure that is used. This stack-based method ensures accurate, linear evaluation in a format appropriate for simple devices. For example, in a postfix expression like $5\ 7\ +$, each number is pushed onto the stack, and when an operator appears, the final two values are popped, computed, and the result is pushed back.

I added sorting algorithms like Bubble Sort, Merge Sort, Quick Sort, and Selection Sort to the arithmetic operation module to expand the Postfix++ calculator's capabilities beyond those specified in the brief. Although evaluating postfix expressions with a stack was the main task, I used these sorting algorithms to show the stack's contents in a sorted order before each arithmetic operation without changing the computation itself. This method helps users better understand how data moves through the stack. For example, when the calculator receives a postfix expression such as $5\ 8\ 3\ +\ *$, it first determines that $8 + 3 = 11$ and then $5 * 11 = 55$. For display purposes only, the current stack $[3, 5, 8]$ is sorted and displayed using the selected algorithm prior to each operation. This feature not only improves the user experience but also demonstrates how sorting techniques can be used practically in a relevant setting.

To store variable assignments such as $A\ 3\ =$, I created a symbol table as required in the brief. A Binary Search Tree that allows sorted insertion, search, and delete. A Direct Addressing Table that uses a fixed array of 26 slots for A to Z to enable $O(1)$ access, making it perfect for systems with limited memory. And a Hash Table that mimics real-world variable lookups with extra features like sorted output using insertion sort are the three data structures I used to design this table. Depending on the requirements of the system, these various implementations serve to demonstrate how various data structures can be improved for simplicity or performance. Along with handling symbols, I used an inorder traversal system that visually reconstructs the corresponding infix expression with full parentheses from a postfix expression, such as $8\ 2\ 6\ +\ *$ to $(8*(2+6))$. This demonstrates the use of traversal algorithms and trees in expression parsing.

My solution is essentially more than just a functional calculator. It is an example of how basic data structures like arrays, stacks, and trees, as well as algorithms like sorting, can be efficiently integrated into a practical interpreter to improve learning and highlight the capabilities of structured programming.

2. A list of the data structures in your solution. Explain why each chosen data structure is suitable for the task.

I used a variety of basic data structures in the Postfix++ design, each one chosen with care to meet the coursework brief's specifications for the calculator, symbol table, and expression handling.

1. Stack

A stack is a linear data structure that adheres to the Last In, First Out (LIFO) principle, which states that elements are added and removed from the same end, or the top of the stack. The stack is the central component of the postfix evaluation logic in my project. All arithmetic evaluation functions use it.

When an operator is encountered, the final two operands are popped, evaluated, and the result is pushed back onto the stack. The stack is where operands are stored while they are read from the expression. The program, for example, evaluates $5 + 7 = 12$ and then $1 * 12 = 12$ in the expression $1\ 5\ 7\ +\ *$ while keeping all intermediate states in the stack. Because of its fundamental alignment with postfix notation, this structure was the most appropriate.

2. Array

A linear data structure called an array uses indices to provide constant-time access to its elements, which are stored at locations in the same memory. In my implementation of Postfix++, arrays played several functions. A direct addressing table that maps each variable from A to Z to a distinct index was created using a fixed-size array of length 26 to ensure quick and memory-efficient lookup. JavaScript's built-in array methods push, pop allowed for stack-like behaviour for expression evaluation, while arrays also served as the foundation for stacks. Additionally, to ensure that the original stack remained unchanged during display operations, temporary arrays were created when sorting stack contents using algorithms like Insertion Sort.

3. Binary Search Tree (BST)

Each node in a binary search tree has a maximum of two children, making it a hierarchical data structure. Whereas the right child stores values higher than the parent, the left child stores values lower than the parent node. When assigning and retrieving variable values (A–Z), my solution uses the module's implementation of the BST to manage the symbol table. The BST stores this assignment in sorted order, for instance, when the expression $A\ 5\ =$ is entered. A user can update a variable, find its value, or remove it from the symbol table with the help of the tree structure's effective search, insert, and delete functions. An inorder traversal to list every variable in alphabetical order is also made possible by this structure.

4. Hash Table

A data structure called a hash table uses a hash function to determine the storage location and maps keys to values. JavaScript frequently uses objects

as dictionaries or hash tables. I created a JavaScript object to implement the symbol table with each key representing a variable name (A–Z) and the value being the assigned number. For storing variable assignments such as $B = 10$, this structure facilitates rapid insertion and retrieval. Later, if the user enters $B + 5$, the interpreter gets the value of B from the hash table, calculates it, and shows the outcome. Dictionary and hash table are frequently used interchangeably in programming contexts, and my implementation reflects this idea.

5. Inorder Transversal

Using an inorder traversal, I implemented this data structure in to transform postfix expressions into fully parenthesized infix expressions. To help users better understand the order of operators and the structure of the original expression, the postfix expression $1\ 5\ 8\ +\ *$, for example, is converted into an expression tree and printed as $(1*(5+8))$. The relationship between tree traversal and expression evaluation is further illustrated in this visualization, which helps to meet the module's learning objectives.

A strong, engaging, and instructive Postfix++ interpreter is produced by combining the following data structures stack, binary search tree, array, hash table, direct addressing table, and inorder transversal.

3. An explanation of the original algorithms of your solution in non-technical language.

Postfix Expression Evaluation

The calculator's primary function is to solve mathematical problems expressed in postfix form, in which operators are written after numbers. The formula $1\ 5\ 8\ +\ *$, for example, instructs the calculator to add 5 and 8 to get 13, then multiply the result by 1 to get 13. One item at a time, the calculator processes the input and stores the numbers as they are entered. Upon encountering a mathematical symbol such as +, it returns to the previous two numbers, computes the outcome, and then stores it once more. The entire expression is solved by repeating this process. The number of parts in the expression determines how long it takes. If there are n items, the time is approximately $O(n)$. To remember every number during computation, $O(n)$ space is also used. This approach is effective due to its simplicity and efficiency.

Sorting Before Evaluation

I included sorting as an extra feature to demonstrate how sorting algorithms operate, even though it is not necessary to solve postfix expressions. The calculator displays the current set of numbers in sorted order using bubble sort, insertion sort, merge sort, quick sort, and selection sort before each calculation step. For example, in $3\ 2\ 5\ +\ *$, the numbers may be displayed sorted as 2 3 5

prior to the addition of 2 and 5. This has no impact on the outcome and is just for display purposes. Sorting small sets of numbers makes it easier to see how each approach works. While merge sort and quick sort take $O(n \log n)$ time, bubble sort, insertion sort, and selection sort take $O(n^2)$ time in the worst case. In practice, they all run quickly because we only sort a few numbers at a time. Since merge sort creates new lists, it uses $O(n)$ space, while most of them use very little extra space ($O(1)$).

Symbol Table using Binary Search Tree

I use a binary search tree to store variable values like $A\ 5 =$. With this arrangement, variables are arranged so that the calculator can quickly locate the correct one by moving left or right based on the letter. When A, B, and C are added together, for example, the calculator puts them inside the tree in alphabetical order. The calculator then looks through the tree to determine A's value when $A\ 2 +$ is entered. For medium-sized variable sets, this approach performs well. If the tree is balanced, finding or storing a variable takes roughly $O(\log n)$. If it is not, it may take up to $O(n)$. Because each variable is stored as a separate component in the tree, its memory usage is $O(n)$.

Symbol Table using Direct Addressing

Direct addressing is another technique I employed to store variables. I made a fixed list of 26 boxes because the only variable names that are allowed are single letters from A to Z. A stored value such as $B\ 7 =$ is put straight into the box for B. The calculator knows exactly where to look if the user later inputs $B\ 3 +$ and no searching is necessary. Finding, adding, or removing values only takes $O(1)$ time with this incredibly quick method. Additionally, it has a fixed memory usage, making its space complexity $O(1)$. As stated in the brief, this is perfect for tiny devices with little storage.

Symbol Table using Hash Table

I also used a hash table, which functions similarly to a dictionary, to implement the symbol table. The calculator uses a unique rule to determine where to store a value such as $C\ 10 =$ and then uses that same rule to retrieve it later. This approach is extremely quick and adaptable. Finding or storing values typically takes $O(1)$ time, but in extremely rare situations, if multiple variables end up in the same location, it may take $O(n)$. The space complexity is $O(n)$ since the memory required increases as the number of variables increases. To make the displayed list of variables easier to read, I also added a small sorting feature using insertion sort. Insertion sort uses very little memory ($O(1)$) and executes quickly with a time complexity of $O(n^2)$ because there are only 26 possible variables.

Inorder Traversal

I included a feature that displays expressions enclosed in brackets to help users in understanding the meaning of their postfix expression in standard math format. $3\ 4\ 5\ +\ *$, for example, becomes $3*(4+5)$. The calculator achieves this by creating a tiny structure that connects numbers and symbols in a manner like a family tree. We refer to this as an expression tree. The calculator builds the bracketed result by reading the postfix expression, building the tree step-by-step, and then walking through it in the proper order. This helps users in understanding the sequence of events. Where n is the number of tokens in the expression, it takes $O(n)$ time and $O(n)$ space to build and print.

In addition to meeting the brief, every component of my solution showed the knowledge I gained from this course. I made sure that every component of the calculator functions properly, even on tiny devices, by considering both time and memory usage.

- 4. Pseudocode for each original algorithm. You are urged to follow the pseudocode conventions of Concern at al, Chapter 2. Pay great attention to how you lay out your pseudocode. In particular, take care of use structured indentation. Pseudocode that is not correctly indented, or is otherwise unreadable, or has been pasted as an image, will not be marked.**

Arithmetic.js

```
// Pseudocode 1
// Sorting 1
function bubble_sort(A)
  sort ← copy of A
  N ← length of sort

  for i ← 0 to N - 2 do
    for j ← 0 to N - i - 2 do
      if sort[j] > sort[j + 1] then
        swap sort[j], sort[j + 1]
      end if
    end for
  end for
  return sort
end function

// Sorting 2
function insertion_sort(A)
  sort ← copy of A
```

```

N ← length of sort

for i ← 1 to N - 1 do
    key ← sort[i]
    j ← i - 1

    while j ≥ 0 and sort[j] > key do
        sort[j + 1] ← sort[j]
        j ← j - 1
    end while

    sort[j + 1] ← key
end for

return sort
end function

// Sorting 3
function merge_sort(A)
    if length of A ≤ 1 then
        return A
    end if

    mid ← floor(length of A / 2)
    small ← MergeSort(A[0 to mid - 1])
    big ← MergeSort(A[mid to end])

    return Merge(small, big)
end function

function merge(left, right)
    result ← empty list
    i ← 0
    j ← 0
    while i < length(left) and j < length(right) do
        if left[i] < right[j] then
            append left[i] to result
            i ← i + 1
        else
            append right[j] to result
            j ← j + 1
        end if
    end while
    append remaining elements of left[i..end] and right[j..end] to result
    return result
end Algorithm

```

```

// Sorting 4
function QuickSort(A)
  if length of A  $\leq$  1 then
    return A
  end if

  pivot  $\leftarrow$  last element of A
  small  $\leftarrow$  empty list
  big  $\leftarrow$  empty list

  for i  $\leftarrow$  0 to length of A - 2 do
    if A[i] < pivot then
      append A[i] to small
    else
      append A[i] to big
    end if
  end for

  return QuickSort(small) + [pivot] + QuickSort(big)
end function

```

```

// Sorting 5
function SelectionSort(array)
  sort  $\leftarrow$  copy of array

  for i  $\leftarrow$  0 to length of sort - 2 do
    min  $\leftarrow$  i

    for j  $\leftarrow$  i + 1 to length of sort - 1 do
      if sort[j] < sort[min] then
        min  $\leftarrow$  j
      end if
    end for

    if min  $\neq$  i then
      swap sort[i] and sort[min]
    end if
  end for

  return sort
end function

```

```

// Postfix Evaluation
function Evaluation_Postfix(tokens, sorting_show, sort_algo)
  stack  $\leftarrow$  empty list

```

```

for each token in tokens do
  if token is a number then
    if sorting_show then
      print "Token", token, "is an operand."
    end if
    push token as number into stack
  else if token is in ["+", "-", "*", "/"] then
    if sorting_show then
      print "Token", token, "is an operator."
    end if

    if length of stack < 2 then
      print "Error: Not enough operands for operator", token
      return
    end if

    if sorting_show then
      print "Before stack: ", stack
      sorted ← sort_algo(copy of stack)
      print "Before", token, "operation, sorted stack:", sorted
    end if

    value_1 ← pop from stack
    value_2 ← pop from stack

    switch token do
      case "+":
        result ← value_2 + value_1
      case "-":
        result ← value_2 - value_1
      case "*":
        result ← value_2 * value_1
      case "/":
        result ← value_2 / value_1
    end switch

    if sorting_show then
      print "Calculation:", value_2, token, value_1, "=", result
    end if

    push result into stack
  else
    print "Error: Invalid token", token
    return
  end if
end for

```



```

    if length of stack = 1 then
        print "Postfix evaluation result:", pop from stack
    else
        print "Error: Invalid postfix expression. Stack:", stack
    end if
end function

```

Inorder_transversal

```

// Pseudocode 2

// Expression Tree
function tree_expression(tokens)
    stack ← empty list

    for each token in tokens do
        if token is a number or valid variable then
            node ← create node with token
            push node into stack
        else if token is a valid operator then
            right ← pop from stack
            left ← pop from stack

            if left is null or right is null then
                print "Error: Not enough operands for operator", token
                return null
            end if

            node ← create node with token
            node.left ← left
            node.right ← right
            push node into stack
        else
            print "Error: Invalid token", token
            return null
        end if
    end for

    if length of stack = 1 then
        return stack[0]
    else
        return null
    end if
end function

// Infix expression with parentheses
function infix_parantheses(node)
    if node is null then

```

```

        return empty string
    end if

    if node.left is null and node.right is null then
        return node.value
    end if

    left ← infix_parantheses(node.left)
    right ← infix_parantheses(node.right)

    return "(" + left + node.value + right + ")"
end function

```

Binary_search

```

// Pseudocode 3

// Recursive function to insert a node
function insert_node(node, key, value)
    if node is null then
        return new Node(key, value)
    end if

    if key < node.key then
        node.left ← Insert_Node(node.left, key, value)
    else if key > node.key then
        node.right ← Insert_Node(node.right, key, value)
    else
        node.value ← value
    end if

    return node
end function

// Recursive function to search for a key
function search_Node(node, key)
    if node is null then
        return null
    end if

    if key = node.key then
        return node.value
    else if key < node.key then
        return Search_Node(node.left, key)
    else
        return Search_Node(node.right, key)
    end if
end function

```

```

// Recursive function to delete node
function delete_Node(node, key)
  if node is null then
    return null
  end if

  if key < node.key then
    node.left ← Delete_Node(node.left, key)
  else if key > node.key then
    node.right ← Delete_Node(node.right, key)
  else
    if node.left is null then
      return node.right
    else if node.right is null then
      return node.left
    end if

    minimum_node ← Minimum_Find(node.right)
    node.key ← minimum_node.key
    node.value ← minimum_node.value
    node.right ← Delete_Node(node.right, minimum_node.key)
  end if

  return node
end function

// Find minimum node
function minimum_find(node)
  while node.left is not null do
    node ← node.left
  end while

  return node
end function

// Recursive function to print key-value in sorted order
function sort_order(node)
  if node is not null then
    Sort_Order(node.left)
    print node.key, "=", node.value
    Sort_Order(node.right)
  end if
end function

// Postfix Evaluation
function evaluate_postfix(expression)

```

```

tokens ← split expression by space
stack ← empty list

for each token in tokens do
  if token is a number then
    push token into stack
  else if token is a valid variable then
    push token into stack
  else if token is an operator then
    if token = "=" then
      if stack has less than 2 elements then
        print "Error: '=' needs variable and value"
        return empty list
      end if

      value ← pop from stack
      variable ← pop from stack

      if value is variable then
        val ← Search_Node(root, value)
      else
        val ← value
      end if

      Insert_Node(root, variable, val)
    else
      if stack has less than 2 elements then
        print "Error: operator requires two operands"
        return empty list
      end if

      b ← pop from stack
      a ← pop from stack

      if a is variable then
        value_A ← Search_Node(root, a)
      else
        value_A ← a
      end if

      if b is variable then
        value_B ← Search_Node(root, b)
      else
        value_B ← b
      end if

      if either value_A or value_B is null then

```

```

        print "Error: undefined variable"
        return empty list
    end if

    result ← apply operator to value_A and value_B
    push result into stack
end if
else
    print "Error: Invalid token"
    return empty list
end if
end for

return stack
end function

```

Direct_addressing

```

// Pseudocode 4

// Convert variable to an index
function variable_to_index(variable)
    return ASCII(variable) - ASCII('A')
end function

// Insert variable into symbol table
function variable_insert(key, value)
    index ← Variable_To_Index(key)
    symbolTable[index] ← value
end function

// Search variable in symbol table
function variable_search(key)
    index ← Variable_To_Index(key)

    if symbolTable[index] ≠ null then
        return symbolTable[index]
    else
        return null
    end if
end function

// Delete variable in symbol table
function variable_delete(key)
    index ← Variable_To_Index(key)
    symbolTable[index] ← null
end function

```

```

// Print symbol table
function symbol_table()
    empty ← true
    print "Symbol Table:"

    for i ← 0 to 25 do
        if symbolTable[i] ≠ null then
            variable ← CHAR(i + ASCII('A'))
            print variable, "=", symbolTable[i]
            empty ← false
        end if
    end for

    if empty = true then
        print "Symbol table is empty."
    end if
end function

// Postfix Evaluation
function evaluate_postfix(expression)
    tokens ← split expression by spaces
    stack ← empty list

    for each token in tokens do
        if token is a number then
            push token to stack
        else if token is a valid variable then
            push token to stack
        else if token is a valid operator then
            if token = '=' then
                if stack has less than 2 items then
                    print "Error: '=' needs a variable and value"
                    return empty list
                end if

                value ← pop from stack
                variable ← pop from stack

                if value is a variable then
                    val ← Variable_Search(value)
                else
                    val ← value
                end if

                Variable_Insert(variable, val)
            end if
        else // +, -, *, /

```

```

    if stack has less than 2 items then
        print "Error: operator needs two operands"
        return empty list
    end if

    b ← pop from stack
    a ← pop from stack

    if a is a variable then
        value_A ← Variable_Search(a)
    else
        value_A ← a
    end if

    if b is a variable then
        value_B ← Variable_Search(b)
    else
        value_B ← b
    end if

    if value_A = null or value_B = null then
        print "Error: Undefined variable"
        return empty list
    end if

    result ← Apply_Operator(value_A, value_B, token)
    push result to stack
end if
else
    print "Error: Invalid token"
    return empty list
end if
end for

return stack
end function

```

Hashtable

```

// Pseudocode 5

// Insert variable and value
function variable_insert(key, value)
    symbolTable[key] ← value
end function

// Search variable and value
function variable_search(key)

```

```

    if key exists in symbolTable then
        return symbolTable[key]
    else
        return null
    end if
end function

// Delete variable and value
function variable_delete(key)
    if key exists in symbolTable then
        remove key from symbolTable
    end if
end function

// Insertion sort
function insertion_sort(array)
    for i ← 1 to length(array) - 1 do
        key ← array[i]
        j ← i - 1

        while j ≥ 0 and array[j] > key do
            array[j + 1] ← array[j]
            j ← j - 1
        end while

        array[j + 1] ← key
    end for

    return array
end function

// Display the sorted symbol table
function symbol_table_sorted()
    keys ← list of keys in symbolTable

    if length of keys = 0 then
        print "Symbol table is empty."
        return
    end if

    sortedKeys ← Insertion_Sort(keys)

    for each key in sortedKeys do
        print key, "=", symbolTable[key]
    end for
end function

```



```

// Postfix Evaluation
function evaluate_postfix(expression)
  tokens ← split expression by spaces
  stack ← empty list

  if length(tokens) = 1 and token is valid variable then
    value ← Variable_Search(tokens[0])
    if value = null then
      print "Error: Variable not found"
      return empty list
    else
      return [value]
    end if
  end if

  for each token in tokens do
    if token is a number then
      push token to stack
    else if token is a valid variable then
      push token to stack
    else if token is an operator then
      if token = '=' then
        if stack has fewer than 2 items then
          print "Error: '=' needs variable and value"
          return empty list
        end if

        value ← pop from stack
        variable ← pop from stack

        if value is a variable then
          val ← Variable_Search(value)
        else
          val ← value
        end if

        Variable_Insert(variable, val)
      else
        if stack has fewer than 2 items then
          print "Error: operator needs two operands"
          return empty list
        end if

        b ← pop from stack
        a ← pop from stack

        if a is a variable then

```

```

        valueA ← Variable_Search(a)
    else
        valueA ← a
    end if

    if b is a variable then
        valueB ← Variable_Search(b)
    else
        valueB ← b
    end if

    if valueA = null or valueB = null then
        print "Error: Undefined variable"
        return empty list
    end if

    result ← Apply_Operator(valueA, valueB, token)
    push result to stack
end if
else
    print "Error: Invalid token"
    return empty list
end if
end for

return stack
end function

```

- 5. Javascript implementations of each of your original algorithms. The implementations should match the algorithm pseudocode of the previous question parts and should be written as Javascript functions. Do not insert images.**

Arithmetic.js

```

// Program 1 : Solving arithmetic operations

// Name: Subathra Sundarbabu
// Student id: 10263704
// Module code: CM2035
// Module name: Algorithms & Data Structures II
// Desc (wat the program does): Calculate Arithmetic Postfix operations and
prints out the sorted stack as well.
//          Sorted stack is only for display purpose and does not affect
the calculations.
// Date: 22 June 2025

```

```

// ///////////////////////////////////
// Sorting 1 : Bubble Sort //
// ///////////////////////////////////
function bubble_sort(array) {
  // Copy of the array so the original calculation is not affected
  let sort = [...array];
  // For loop
  for (let i = 0; i < sort.length - 1; i++) {
    // Nested for loop to compare the numbers
    for (let j = 0; j < sort.length - i - 1; j++) {
      // If the current number is bigger than the next one, swap
      if (sort[j] > sort[j + 1]) {
        [sort[j], sort[j + 1]] = [sort[j + 1], sort[j]];
      }
    }
  }
  // Return the sorted array
  return sort;
}

```

```

// ///////////////////////////////////
// Sorting 2 : Insertion Sort //
// ///////////////////////////////////

function insertion_sort(array) {
  // Copy of the array so the original calculation is not affected
  let sort = [...array];
  // For loop, start from the second number
  for (let i = 1; i < sort.length; i++) {
    // Store the current number
    let key = sort[i];

    // Compare with the number before it
    let j = i - 1;

    // Move numbers which are bigger than the key
    while (j >= 0 && sort[j] > key) {
      // Move the bigger number to the right
      sort[j + 1] = sort[j];
      // Move to the previous number
      j = j - 1;
    }
    // Put the key at the sorted position
    sort[j + 1] = key;
  }
  // Return the sorted array
}

```

```

    return sort;
}

//////////
// Sorting 3 : Merge Sort //
//////////
function merge_sort(array) {
    // If the array has 0 or 1 number, it is sorted
    if (array.length <= 1) return array;

    // Find the middle index
    const mid = Math.floor(array.length / 2);

    // Sort the left half
    const small = merge_sort(array.slice(0, mid));

    // Sort the right half
    const big = merge_sort(array.slice(mid));

    // Merge the left and right half
    return merge(small, big);
}

// Merge the 2 sorted array into 1
function merge(small, big) {
    // Empty array to store the result
    let result = [];

    // Pointer
    let i = 0;

    // Pointer
    let j = 0;

    // Compare numbers until one array is completed
    while (i < small.length && j < big.length) {
        // If the current number in left is smaller, add it to result
        if (small[i] < big[j]) {
            // Move pointer forward
            result.push(small[i++]);
        } else {
            // Move pointer forward
            result.push(big[j++]);
        }
    }

    // Leftover numbers add to result
    return result.concat(small.slice(i)).concat(big.slice(j));
}

```

```

}

//////////
// Sorting 4: Quick sort //
//////////
function quick_sort(array) {
  // If the array has 0 or 1 number, it is sorted
  if (array.length <= 1) return array;

  // Last number is pivot
  const pivot = array[array.length - 1];

  // empty array to store small values
  const small = [];

  // empty array to store big values
  const big = [];

  // For loop go into all numbers except the pivot
  for (let i = 0; i < array.length - 1; i++) {
    // If the current number is smaller than the pivot, put it in left
    if (array[i] < pivot) small.push(array[i]);
    // If the current number is bigger than the pivot, put it in right
    else big.push(array[i]);
  }
  // Sort and return the combined array, pivot in middle
  return [...quick_sort(small), pivot, ...quick_sort(big)];
}

//////////
// Sorting 5: Selection sort //
//////////
function selection_sort(array) {
  // Copy of the array so the original calculation is not affected
  let sort = [...array];

  // Loop thorough each number in the array, expect the last one
  for (let i = 0; i < sort.length - 1; i++) {
    // Assume the current index is the smallest number
    let min = i;

    // Look for a smaller value
    for (let j = i + 1; j < sort.length; j++) {
      // If there is, remember the index
      if (sort[j] < sort[min]) min = j;
    }
    // If a smallest number is found, swap

```

```

    if (min !== i) {
        // Swap the numbers
        [sort[i], sort[min]] = [sort[min], sort[i]];
    }
}
// Return the sorted array
return sort;
}

// Postfix evaluation
function Evaluation_Postfix(tokens, sorting_show, sort_algo) {
    // Stack to store the numbers
    let stack = [];

    // Go into each postfix expressions
    for (let token of tokens) {
        // If the token is a number
        if (!isNaN(token)) {
            // If the user wants details
            if (sorting_show){
                // Console log statement
                console.log(`Token '${token}' is an operand.`);
            }
            // Push the number into the stack
            stack.push(Number(token));
        }
        // If the token is operator
        else if (["+", "-", "*", "/"].includes(token)) {
            // If the user want details
            if (sorting_show) {
                // Console log statement
                console.log(`Token '${token}' is an operator.`);
            }

            // if less than 2 numbers to apply operator
            if (stack.length < 2) {
                // Console log statement error message
                console.log(`Error: Not enough operands for operator '${token}'`);
                return;
            }

            // If the user want details
            if (sorting_show) {
                // Console log before stack
                console.log(`\nBefore stack:    ', [...stack]);
                // Apply the sorting function
                const sorted = sort_algo([...stack]);
            }

```

```

        // Console log the sorted stack
        console.log(` Before '${token}' operation, sorted stack:`, sorted);
    }

    // Take the last 2 numbers from the stack
    const value_1 = stack.pop();
    const value_2 = stack.pop();
    let result;

    // Math operations for the different operators
    switch (token) {
        case '+': result = value_2 + value_1; break; // addition
        case '-': result = value_2 - value_1; break; // subtraction
        case '*': result = value_2 * value_1; break; // multiplication
        case '/': result = value_2 / value_1; break; // division
    }

    // If the user wants details
    if (sorting_show) {
        // Console log statement
        console.log(` Calculation:      ${value_2} ${token} ${value_1} =
${result}\n`);
    }

    // Push result back to the stack
    stack.push(result);
}
// If the token is not a number or operator
else {
    // Console log statement error message
    console.log(` Error: Invalid token '${token}'`);
    return;
}
}

// Only one number as answer
if (stack.length === 1) {
    // Console log statement the output
    console.log("\nPostfix evaluation result:", stack.pop());
}
// More than 1 number present in output
else {
    // Console log statement error message
    console.log("Error: Invalid postfix expression. Stack:", stack);
}
}

```

```

// Menu for the sorting algos
function Menu_Sort(question, back_menu, algo_name, sort_algo) {
  // Shows the menu
  function options_ask() {
    // Console log statements for the menu options
    console.log("\n=====");
    console.log(`    ${algo_name} Sort Postfix Options`);
    console.log("=====\\n");
    console.log("1. Detailed Output (Sorting + Symbol Lookup)");
    console.log("2. Final Output");
    console.log("3. Back to Menu\\n");

    // Question to ask the user to input option 1 to 3
    question.question("Choose an option (1-3): ", (option) => {
      // If the user input option 3
      if (option === "3") {
        // Return back to menu
        back_menu();
        return;
      }

      // If the user input is not 1,2, or 3
      if (option !== "1" && option !== "2") {
        // Console log statement error message
        console.log("Invalid choice. Please enter 1, 2, or 3.\\n");
        // Ask the user to input again
        options_ask();
        return;
      }

      // If the user select 1, show the details
      const sorting_show = option === "1";

      // Question to ask user to input the postfix expression
      question.question("Enter space-separated postfix expression (e.g. 2 3 1 *
+ 9 -): ", (input) => {
        // If got spaces in input
        if (!input.includes(" ")) {
          // Console log statement error message to input without spaces
          console.log("Error: Please enter a space-separated postfix
expression.");
          // Return back to menu
          back_menu();
          return;
        }

        // Break the input into tokens

```



```

        const tokens = input.trim().split(/\s+/);
        // Evaluate the expressions
        Evaluation_Postfix(tokens, sorting_show, sort_algo);
        // Return back to menu
        back_menu();
    });
});
}

// Ask the user input
options_ask();
}

// Exported functions to use in main.js
module.exports = {
    // Bubble sort
    bubblesort: (question, back_menu) => Menu_Sort(question, back_menu,
    "Bubble", bubble_sort),
    // Insertion sort
    insertionsort: (question, back_menu) => Menu_Sort(question, back_menu,
    "Insertion", insertion_sort),
    // Merge sort
    mergesort: (question, back_menu) => Menu_Sort(question, back_menu,
    "Merge", merge_sort),
    // quick sort
    quicksort: (question, back_menu) => Menu_Sort(question, back_menu,
    "Quick", quick_sort),
    // selection sort
    selectionsort: (question, back_menu) => Menu_Sort(question, back_menu,
    "Selection", selection_sort),
};

```

Inorder_transversal.js

```

// Program 2 : Inorder_transversal

// Name: Subathra Sundarbabu
// Student id: 10263704
// Module code: CM2035
// Module name: Algorithms & Data Structures II
// Desc (wat the program does): Shows the infix parantheses for the user input
postfix evaluation
// Date: 22 June 2025

// Create Node
function node_creating(value) {
    return {

```

```

    // Node value
    value: value,
    // left child
    left: null,
    // right child
    right: null
  };
}

// To check is the operator is valid
function operator_valid(token) {
  // Only these operators are valid
  return ['+', '-', '*', '/', '='].includes(token);
}

// To check is the variable is valid
function variable_valid(token) {
  // Only A - Z is valid
  return token.length === 1 && token >= 'A' && token <= 'Z';
}

// Expression Tree
function tree_expression(tokens) {
  // Stack to store nodes
  const stack = [];

  // Loop thorough each variable
  for (const token of tokens) {
    // If token is not null and is valid variable
    if (!isNaN(token) || variable_valid(token)) {
      // Push to the stack
      stack.push(node_creating(token));
    }
    // If it is a operator
    else if (operator_valid(token)) {
      // Pop 2 nodes from the stack
      const right = stack.pop();
      const left = stack.pop();

      // Not enough operands
      if (!left || !right) {
        // Console log statement error message
        console.log(`Error: Not enough operands for operator '${token}'.`);
      }

      // Return null
      return null;
    }
  }
}

```

```

        // New operator node
        const node = node_creating(token);
        // Left node
        node.left = left;
        // Right node
        node.right = right;
        // Push new node to the stack
        stack.push(node);
    }
    // If the token not valid
    else {
        // Console log statement error message
        console.log(`Error: Invalid token '${token}'.`);
        return null;
    }
}
// If one node is left
if (stack.length === 1) {
    // return stack at index 0
    return stack[0];
} else {
    // else return null
    return null;
}
}

// Infix expression with parentheses
function infix_parentheses(node) {
    // Empty string if the node is null
    if (!node) return "";

    // Return the value if the no left and right node
    if (!node.left && !node.right) {
        return node.value;
    }

    // Process left and wrap with parentheses
    const left = infix_parentheses(node.left);

    // Process right and wrap with parentheses
    const right = infix_parentheses(node.right);

    // Return the combined string
    return `(${left}${node.value}${right})`;
}

```

```

// Expand postfix expression
function postfix_expression(expression) {
    // split tokens by space
    const tokens = expression.trim().split(/\s+/);
    // build expression tree
    const root_tree = tree_expression(tokens);

    // If tree is invalid
    if (!root_tree) {
        // Console log statement error message
        console.log("Invalid postfix expression.");
        return;
    }

    // Index expression
    const infix = infix_parentheses(root_tree);
    // Console log statement with the output
    console.log("Expanded (Infix with parentheses):", infix);
}

// Menu for inorder transversal
function inorder_traversal(question, back_menu) {
    // Menu options
    function menu_inorder() {
        // Console log statements for the options
        console.log("\n=====");
        console.log("    Inorder Traversal Expansion Menu    ");
        console.log("=====\\n");
        console.log("1. Enter a postfix expression to expand");
        console.log("2. Back to Symbol Table Menu\\n");
        // Question for the user for the input
        question.question("Enter option (1-2): ", option_handle);
    }

    // Handle the options
    function option_handle(option) {
        // Trim any spaces
        switch (option.trim()) {
            // Case 1
            case '1':
                // Console log statement
                console.log("Enter a postfix expression (type 'menu' to return):");
                // Input mode
                input_mode();
                break;
            // Case 2
            case '2':

```

```

        // Back to symbol table menu
        back_menu();
        break;
    default:
        // Console log statement for invalid option
        console.log("Invalid option. Please enter 1 or 2.");
        // show menu again
        menu_inorder();
    }
}

// Allow multiple inputs from the user
function input_mode() {
    // show > as starting the console
    question.question('> ', (line) => {
        // If the user types menu
        if (line.trim().toLowerCase() === 'menu') {
            // Go back to the menu
            menu_inorder();
            return;
        }
        // Show the infix form
        postfix_expression(line);

        // ask for the next input
        input_mode();
    });
}

// Start the menu
menu_inorder();
}

// Export function to use in main.js
module.exports = inorder_traversal;

```

Binary_search.js

```

// Program 3 : Binary Search

// Name: Subathra Sundarbabu
// Student id: 10263704
// Module code: CM2035
// Module name: Algorithms & Data Structures II
// Desc (wat the program does): Binary Search algorithm on symbol table
// Date: 22 June 2025

```

```

// Create Node
function node_creating(key, value) {
  return {
    // Key value
    key: key,
    // Node value
    value: value,
    // left child
    left: null,
    // right child
    right: null
  };
}

// root of binary search
let root = null;

// To check if the operator is valid
function operator_valid(token) {
  // Only these operators are valid
  return ['+', '-', '*', '/', '='].includes(token);
}

// To check if the variable is valid
function variable_valid(token) {
  // Only A - Z is valid
  return token.length === 1 && token >= 'A' && token <= 'Z';
}

// Insert variable and value
function variable_insert(key, value) {
  root = insert_node(root, key, value);
}

// Recursive function to insert a node
function insert_node(node, key, value) {
  // If no node, create one
  if (node === null) return node_creating(key, value);
  // Insert to the left if smaller
  if (key < node.key) node.left = insert_node(node.left, key, value);
  // Insert to the right if bigger
  else if (key > node.key) node.right = insert_node(node.right, key, value);
  // If key exists, update value
  else node.value = value;
  return node;
}

```

```

// Search for variable by key
function variable_search(key) {
    return search_node(root, key);
}

// Recursive function to search for a key
function search_node(node, key) {
    // Not found return null
    if (node === null) return null;
    // If exists, return the value
    if (key === node.key) return node.value;
    // Search left
    if (key < node.key) return search_node(node.left, key);
    // Search right
    return search_node(node.right, key);
}

// delete variable by key
function variable_delete(key) {
    root = delete_node(root, key);
}

// Recursive function to delete node
function delete_node(node, key) {
    // If tree is empty, return null
    if (node === null) return null;
    // Search left
    if (key < node.key) node.left = delete_node(node.left, key);
    // Search right
    else if (key > node.key) node.right = delete_node(node.right, key);
    // Found the node to delete
    else {
        // If only right
        if (node.left === null) return node.right;
        // If only left
        if (node.right === null) return node.left;
        // Get the smallest node in right
        let minimum_node = minimum_find(node.right);
        // Replace current node with minimum_node key
        node.key = minimum_node.key;
        // Replace the value
        node.value = minimum_node.value;
        // Delete duplicate
        node.right = delete_node(node.right, minimum_node.key);
    }
    // return node
    return node;
}

```

```

}

// Find minimum node
function minimum_find(node) {
  // Left till end
  while (node.left !== null) node = node.left;
  return node;
}

// Print the symbol table
function symbol_table() {
  // If no value in symbol table
  if (root === null) {
    // Console log statement
    console.log("Symbol table is empty.");
  }
  // If something in symbol table
  else {
    // Console log statement
    console.log("Symbol Table:");
    // Order the symbol table values
    sort_order(root);
  }
}

// Recursive function to print key-value in sorted order
function sort_order(node) {
  // If node is not null
  if (node !== null) {
    // Go left
    sort_order(node.left);
    // Print current node
    console.log(` ${node.key} = ${node.value} `);
    // Go right
    sort_order(node.right);
  }
}

// Postfix Evaluation
function evaluate_postfix(expression) {
  // Split input by space
  const tokens = expression.trim().split(' ');
  // Stack to hold the variables
  const stack = [];

  // Loop through each token
  for (const token of tokens) {

```



```

// If token is not null
if (!isNaN(token)) {
    // Push number to stack
    stack.push(Number(token));
}
// If token is a variable
else if (variable_valid(token)) {
    // Push variable to stack
    stack.push(token);
}
// If token is a operator
else if (operator_valid(token)) {
    // if token equal to =
    if (token === '=') {
        // if stack length is less than 2
        if (stack.length < 2) {
            // Console log statement error message
            console.log("Error: '=' operator requires a variable and a value.");
            // Return error
            return [];
        }

        // pop value to assign
        const value = stack.pop();
        // pop variable to assign
        const variable = stack.pop();

        // if variable name is not valid
        if (!variable_valid(variable)) {
            // Console log statement error message
            console.log("Error: Only uppercase variable names A-Z are
accepted.");
            // Return error
            return [];
        }

        // If value is variable name
        if (typeof value === 'string') {
            // Search value
            val = variable_search(value);
        } else {
            // Use number
            val = value;
        }
        // Store value in symbol table
        variable_insert(variable, val);
    } else {

```

```

// if stack length is less than 2
if (stack.length < 2) {
    // Console log statement error message
    console.log(` Error: '${token}' operator requires two operands.`);
    // Return error
    return [];
}

// Pop right operand
const b_value = stack.pop();
// Pop left operand
const a_value = stack.pop();

// If a is a string
if (typeof a_value === 'string') {
    // Search value in symbol table
    value_A = variable_search(a_value);
} else {
    // if not a string, assign is directly
    value_A = a_value;
}

// If b is a string
if (typeof b_value === 'string') {
    // Seach value in symbol table
    value_B = variable_search(b_value);
} else {
    // If not a string, assign it directly
    value_B = b_value;
}

// If a and b are null
if (value_A === null || value_B === null) {
    // Console log statement error message
    console.log("Error: Undefined variable used.");
    // return error
    return [];
}
// Push result back to stack
stack.push(math_evaluation(value_A, value_B, token));
} else {
    // Console log statament for invalid token
    console.log("Error: Invalid token or only uppercase variable names A-Z
are allowed.");
    // Return error
    return [];
}

```

```

    }
}

// Return stack
return stack;
}

// Math operations
function math_evaluation(a, b, operator) {
    switch (operator) {
        case '+': return a + b; // addition
        case '-': return a - b; // subtraction
        case '*': return a * b; // multiplication
        case '/': return a / b; //division
    }
}

// Menu for binary search
function binary_search(question, back_menu) {
    function menu_binarysearch() {
        // Console log statements for the options
        console.log("\n=====");
        console.log("    Binary Search Symbol Table Menu    ");
        console.log("=====\\n");
        console.log("1. Enter a postfix++ expression");
        console.log("2. View the symbol table");
        console.log("3. Search for variable in symbol table");
        console.log("4. Delete variable from symbol table");
        console.log("5. Back to Symbol Table Menu\\n");
        // Question for the user for the input
        question.question("Enter option (1-5): ", option_handle);
    }

    // Handle the options
    function option_handle(option) {
        // Trim any spaces
        switch (option.trim()) {
            // Case 1
            case '1':
                // Console log statement
                console.log("Enter a postfix++ expression (type 'menu' to return):");
                // Input mode
                input_mode();
                break;
            // Case 2
            case '2':
                // Print the symbol table

```

```

symbol_table();
// show the menu again
menu_binarysearch();
break;
// Case 3
case '3':
    // Question for the user to search a variable
    question.question("Search variable (A–Z): ", (input) => {
        // Trim any spaces
        const variable = input.trim();
        // If variable is not valid
        if (!variable_valid(variable)) {
            // Console log statement error message
            console.log("Error: Only uppercase variable names A–Z are
accepted.");
        }
        // If variable is valid
        else {
            // search the value
            const value = variable_search(variable);
            // If value not found
            if (value === null) {
                // Console log statement
                console.log(` Variable '${variable}' not found in symbol table.` );
            }
            // Else if the value is found
            else {
                // Console log statement
                console.log(` Variable '${variable}' has value: ${value} `);
            }
        }
        // Show the menu
        menu_binarysearch();
    });
    break;
// Case 4
case '4':
    // Question for user input to delete variable
    question.question("Delete Variable (A–Z): ", (input) => {
        // Trim any spaces
        const variable = input.trim();
        // If variable is not valid
        if (!variable_valid(variable)) {
            // Console log the error message
            console.log("Error: Only uppercase variable names A–Z are
accepted.");
        }
    }

```

```

        // Else if the variable is valid
        else {
            // Search for the value
            const value = variable_search(variable);
            // If value is not found
            if (value === null) {
                // Console log statement
                console.log(` Variable '${variable}' not found in symbol table.` );
            }
            // If the value is found
            else {
                // delete the variable from symbol table
                variable_delete(variable);
                // Console log statement
                console.log(` Variable '${variable}' deleted from symbol table.` );
            }
        }
        // Show the menu
        menu_binarysearch();
    });
    break;
// Case 5
case '5':
    // Back to symbol table menu
    back_menu();
    break;
default:
    // Console log statement for invalid option
    console.log("Invalid option. Please enter 1-5.");
    // show menu again
    menu_binarysearch();
}
}

// Allow multiple inputs from the user
function input_mode() {
    // show > as starting the console
    question.question('> ', (line) => {
        // if the user types menu
        if (line.trim().toLowerCase() === 'menu') {
            // Go back to the menu
            menu_binarysearch();
            return;
        }
        // Evaluate the postfix expression
        const result = evaluate_postfix(line);
        // Console log the result

```

```

        console.log("Result:", result);
        // Ask for the next input
        input_mode();
    });
}

// Start the menu
menu_binarysearch();
}

// Export function to use in main.js
module.exports = binary_search;

```

direct_addressing.js

```

// Program 4 : Direct addressing

// Name: Subathra Sundarbabu
// Student id: 10263704
// Module code: CM2035
// Module name: Algorithms & Data Structures II
// Desc (wat the program does): Direct addressing algorithm on symbol table
// Date: 22 June 2025

// Table for variables A-Z
let symbolTable = new Array(26).fill(null);

// To check if the operator is valid
function operator_valid(token) {
    // Only these operators are valid
    return ['+', '-', '*', '/', '='].includes(token);
}

// To check if the variable is valid
function variable_valid(token) {
    // Only A -Z is valid
    return token.length === 1 && token >= 'A' && token <= 'Z';
}

// Convert variable to an index
function variable_to_index(variable) {
    // Return the index number
    return variable.charCodeAt(0) - 'A'.charCodeAt(0);
}

// Insert variable into symbol table
function variable_insert(key, value) {

```

```

// Get index from variable name
const index = variable_to_index(key);
// Store the value at the index
symbolTable[index] = value;
}

// Search variable in symbol table
function variable_search(key) {
  // Get the index for the variable
  const index = variable_to_index(key);

  // if the symbol table has value at index
  if (symbolTable[index] !== null) {
    // Return the value if exists
    return symbolTable[index];
  } else {
    // Return null if not exist
    return null;
  }
}

// Delete variable in symbol table
function variable_delete(key) {
  // Get index for the variable
  const index = variable_to_index(key);
  // Set the index to null
  symbolTable[index] = null;
}

// Print symbol table
function symbol_table() {
  // Check if symbol table is empty
  let empty = true;
  // console log statement
  console.log("Symbol Table:");
  // Go thorough all indexes
  for (let i = 0; i < 26; i++) {
    // If value exists at index
    if (symbolTable[i] !== null) {
      // Convert index back to variable name
      const variable = String.fromCharCode(i + 'A'.charCodeAt(0));
      // Print variable and the value
      console.log(` ${variable} = ${symbolTable[i]} `);
      // Mark table as not empty
      empty = false;
    }
  }
}

```

```

// If empty
if (empty) {
  // Console log statement
  console.log("Symbol table is empty.");
}
}

// Postfix Evaluation
function evaluate_postfix(expression) {
  // Split input by space
  const tokens = expression.trim().split(' ');

  // Stack to hold the variables
  const stack = [];

  // Loop through each token
  for (const token of tokens) {
    // If token is not null
    if (!isNaN(token)) {
      // Push number to stack
      stack.push(Number(token));
    }
    // If token is a variable
    else if (variable_valid(token)) {
      // Push variable to stack
      stack.push(token);
    }
    // If token is an operator
    else if (operator_valid(token)) {
      // If token equal to =
      if (token === '=') {
        // if stack length is less than 2
        if (stack.length < 2) {
          // Console log statement error message
          console.log("Error: '=' operator requires a variable and a value.");
          // return error
          return [];
        }
      }

      // pop value to assign
      const value = stack.pop();
      // pop variable to assign
      const variable = stack.pop();

      // if variable name is not valid
      if (!variable_valid(variable)) {
        // Console log statement error message

```



```

        console.log("Error: Only uppercase variable names A-Z are
accepted.");
        // Return error
        return [];
    }

    // If value is vaariable name
    if (typeof value === 'string') {
        // Seach value
        val = variable_search(value);
    } else {
        // Use number
        val = value;
    }
    // Store value in symbol table
    variable_insert(variable, val);
} else {
    // if stack length is less than 2
    if (stack.length < 2) {
        // Console log statement error message
        console.log(` Error: '${token}' operator requires two operands.`);
        // Return error
        return [];
    }

    // Pop right operand
    const b_value = stack.pop();
    // Pop left operand
    const a_value = stack.pop();
    // If a is a string
    if (typeof a_value === 'string') {
        // Search value in symbol table
        value_A = variable_search(a_value);
    } else {
        // if not a string, assign is directly
        value_A = a_value;
    }

    // If b is a string
    if (typeof b_value === 'string') {
        // Seach value in symbol table
        value_B = variable_search(b_value);
    } else {
        // If not a string, assign it directly
        value_B = b_value;
    }
}

```

```

        // If a and b are null
        if (value_A === null || value_B === null) {
            // Console log statement error message
            console.log("Error: Undefined variable used.");
            // return error
            return [];
        }
        // Push result back to stack
        stack.push(math_evaluation(valA, valB, token));
    }
} else {
    // Console log statement for invalid token
    console.log("Error: Invalid token or only uppercase variable names A-Z
are allowed.");
    // Return error
    return [];
}
}

// Return stack
return stack;
}

// Math operations
function math_evaluation(a, b, operator) {
    switch (operator) {
        case '+': return a + b; // addition
        case '-': return a - b; // subtraction
        case '*': return a * b; // multiplication
        case '/': return a / b; // division
    }
}

// Menu for direct addressing
function direct_addressing(question, back_menu){
    function menu_directaddressing() {
        // Console log statements for the options
        console.log("\n=====");
        console.log("    Direct Addressing Symbol Table Menu    ");
        console.log("=====\\n");
        console.log("1. Enter a postfix++ expression");
        console.log("2. View the symbol table");
        console.log("3. Search for variable in symbol table");
        console.log("4. Delete variable from symbol table");
        console.log("5. Back to Symbol Table Menu\\n");
        // Question for the user for the input
        question.question("Enter option (1-5): ", option_handle);
    }
}

```

```

}

// Handle the options
function option_handle(option) {
  // Trim any spaces
  switch (option.trim()) {
    // Case 1
    case '1':
      // Console log statement
      console.log("Enter a postfix++ expression (type 'menu' to return:");
      // Input mode
      input_mode();
      break;
    // Case 2
    case '2':
      // Print the symbol table
      symbol_table();
      // show the menu again
      menu_directaddressing();
      break;
    // Case 3
    case '3':
      // Question for the user to search a variable
      question.question("Search variable (A-Z): ", (input) => {
        // Trim any spaces
        const variable = input.trim();
        // If variable is not valid
        if (!isValidVariable(variable)) {
          // Console log statement error message
          console.log("Error: Only uppercase variable names A-Z are
accepted.");
        }
        // If variable is valid
        else {
          // Search the value
          const value = variable_search(variable);
          // If value not found
          if (value === null) {
            // Console log statement
            console.log(` Variable '${variable}' not found in symbol table. `);
          }
          // Else if the value is found
          else {
            // Console log statement
            console.log(` Variable '${variable}' has value: ${value} `);
          }
        }
      })
  }
}

```

```

        // Show the menu
        menu_directaddressing();
    });
    break;
// Case 4
case '4':
    // Question for user input to delete variable
    question.question("Delete Variable (A–Z): ", (input) => {
        // Trim any spaces
        const variable = input.trim().toUpperCase();
        // If variable is not valid
        if (!isValidVariable(variable)) {
            // Console log the error message
            console.log("Error: Only single uppercase variable names A–Z are
accepted.");
        }
        // Else if the variable is valid
        else {
            // search for the value
            const index = variable_to_index(variable);
            // If value is not found
            if (symbolTable[index] === null) {
                // Console log statement
                console.log(` Variable '${variable}' not found in symbol table.` );
            }
            // If the value is found
            else {
                // delete variable from the symbol table
                variable_delete(variable);
                // Console log statement
                console.log(` Variable '${variable}' deleted from symbol table.` );
            }
        }
    });
    // Show the menu
    menu_directaddressing();
    break;
// Case 5
case '5':
    // Back to symbol table menu
    back_menu();
    break;
default:
    // Console log statement for invalid options
    console.log("Invalid option. Please enter 1-5.");
    // Show menu again
    menu_directaddressing();

```

```

    }
}

// Allow multiple inputs from the user
function input_mode() {
    // show > as starting the console
    question.question('> ', (line) => {
        // if the user types menu
        if (line.trim().toLowerCase() === 'menu') {
            // Go back to the menu
            menu_directaddressing();
            return;
        }
        // Evaluate the postfix expression
        const result = evaluate_postfix(line);
        // Console log the result
        console.log("Result:", result);
        // Ask for the next input
        input_mode();
    });
}

// Start the menu
menu_directaddressing();
}

// Export function to use in main.js
module.exports = direct_addressing;

```

hashtable.js

```

// Program 5 : Hash table

// Name: Subathra Sundarbabu
// Student id: 10263704
// Module code: CM2035
// Module name: Algorithms & Data Structures II
// Desc (wat the program does): Hash table algorithm on symbol table
// Date: 22 June 2025

// Hash table
let symbolTable = {};

// To check if the operator is valid
function operator_valid(token) {
    // Only these operators are valid
    return ['+', '-', '*', '/', '='].includes(token);
}

```

```

}

// To check if the variable is valid
function variable_valid(token) {
    // Only A - Z is valid
    return token.length === 1 && token >= 'A' && token <= 'Z';
}

// Insert variable and value
function variable_insert(key, value) {
    // Add or update the variable
    symbolTable[key] = value;
}

// Search variable and value
function variable_search(key) {
    // Check if the symbol table has the key
    if (symbolTable.hasOwnProperty(key)) {
        // If the key exists, return value
        return symbolTable[key];
    } else {
        // Doesn't exist, return null
        return null;
    }
}

// Delete variable and value
function variable_delete(key) {
    // Only delete if exist
    if (symbolTable.hasOwnProperty(key)) {
        // Remove key value pair from the table
        delete symbolTable[key];
    }
}

function insertion_sort(array) {
    // For loop, start from the second number
    for (let i = 1; i < array.length; i++) {
        // Store the current number
        let key = array[i];

        // Compare with the number before it
        let j = i - 1;

        // Move numbers which are bigger than the key
        while (j >= 0 && array[j] > key) {
            // Move the bigger number to the right

```

```

        array[j + 1] = array[j];
        // Move to the previous number
        j = j - 1;
    }
    // Put the key at the sorted position
    array[j + 1] = key;
}
// Return the sorted array
return array;
}

// Display the sorted symbol table
function symbol_table_sorted() {
    // Get variable names from the symbol table
    const keys = Object.keys(symbolTable);
    // If there are no keys
    if (keys.length === 0) {
        // console log statement. Table is empty
        console.log("Symbol table is empty.");
        return;
    }

    // Sort the keys using insertion sort
    const sortedKeys = insertion_sort(keys);

    // Console log statement
    console.log("Symbol Table (Sorted by variable name):");

    // For loop to sort keys
    for (const key of sortedKeys) {
        // Console log statement to display variable with value
        console.log(` ${key} = ${symbolTable[key]} `);
    }
}

// Postfix Evaluation
function evaluate_postfix(expression) {
    // Split input by space
    const tokens = expression.trim().split(' ');
    // Stack to hold the variables
    const stack = [];

    // Check if the expression contains 1 token and variable is valid
    if (tokens.length === 1 && variable_valid(tokens[0])) {
        // Get the value from the symbol table
        const value = variable_search(tokens[0]);
    }
}

```

```

// If variable is not found
if (value === null) {
    // Console log statement error message
    console.log(` Error: Variable '${tokens[0]}' not found in symbol table.` );
    // Returns error
    return [];
}
// If variable exists
else {
    // Return as an array
    return [value];
}
}

// Loop through each token
for (const token of tokens) {
    // If token is not null
    if (!isNaN(token)) {
        // Push number to stack
        stack.push(Number(token));
    }
    // If token is a variable
    else if (variable_valid(token)) {
        // Push variable to stack
        stack.push(token);
    }
    // If token is a operator
    else if (operator_valid(token)) {
        // If token equal to =
        if (token === '=') {
            // If stack length is less than 2
            if (stack.length < 2) {
                // Console log statement error message
                console.log("Error: '=' operator requires a variable and a value.");
                // return error
                return [];
            }

            // pop value to assign
            const value = stack.pop();
            // pop variable to assign
            const variable = stack.pop();

            // if variable name is not valid
            if (!variable_valid(variable)) {
                // Console log statement error message

```



```

        console.log("Error: Only uppercase variable names A-Z are
accepted.");
        // Return error
        return [];
    }

    // If value is variable name
    if (typeof value === 'string') {
        // Search value
        val = variable_search(value);
    } else {
        // Use number
        val = value;
    }
    // Store value in symbol table
    variable_insert(variable, val);
} else {
    // if stack length is less than 2
    if (stack.length < 2) {
        // Console log statement error message
        console.log(` Error: '${token}' operator requires two operands.`);
        // return error
        return [];
    }
    // Pop right operand
    const b_value = stack.pop();
    // Pop left operand
    const a_value = stack.pop();

    // If a is a string
    if (typeof a_value === 'string') {
        // Search value in symbol table
        value_A = variable_search(a_value);
    } else {
        // if not a string, assign is directly
        value_A = a_value;
    }

    // If b is a string
    if (typeof b_value === 'string') {
        // Search value in symbol table
        value_B = variable_search(b_value);
    } else {
        // If not a string, assign it directly
        value_B = b_value;
    }
}

```

```

        // If a and b are null
        if (value_A === null || value_B === null) {
            // Console log statement error message
            console.log("Error: Undefined variable used.");
            // return error
            return [];
        }
        // Push result back to stack
        stack.push(math_evaluation(value_A, value_B, token));
    }
} else {
    // Console log statament for invalid token
    console.log("Error: Invalid token or only uppercase variable names A-Z
are allowed.");
    // Return error
    return [];
}
}

// Return stack
return stack;
}

// Math operations
function math_evaluation(a, b, operator) {
    switch (operator) {
        case '+': return a + b; // addition
        case '-': return a - b; // subtraction
        case '*': return a * b; // multiplication
        case '/': return a / b; //division
    }
}

// Menu for hash table
function hash_table(question, back_menu) {
    function menu_hashtable() {
        // Console log statements for the options
        console.log("\n=====");
        console.log("    Hash Table Symbol Table Menu    ");
        console.log("=====\\n");
        console.log("1. Enter a postfix++ expression");
        console.log("2. View the symbol table");
        console.log("3. Search for variable in symbol table");
        console.log("4. Delete variable from symbol table");
        console.log("5. Back to Symbol Table Menu\\n");
        // Question for the user for the input
        question.question("Enter option (1-5): ", option_handle);
    }
}

```

```

}

// Handle the options
function option_handle(option) {
  // Trim any spaces
  switch (option.trim()) {
    // Case 1
    case '1':
      // Console log statement
      console.log("Enter a postfix++ expression (type 'menu' to return:");
      // Input mode
      input_mode();
      break;
    // Case 2
    case '2':
      // Print the symbol table
      symbol_table_sorted();
      // show the menu again
      menu_hashtable();
      break;
    // Case 3
    case '3':
      // Question for the user to search a variable
      question.question("Search variable (A-Z): ", (input) => {
        // Trim any spaces
        const variable = input.trim();
        // If variable is not valid
        if (!variable_valid(variable)) {
          // Console log statement error message
          console.log("Error: Only uppercase variable names A-Z are
accepted.");
        }
        // If variable is valid
        else {
          // search the value
          const value = variable_search(variable);
          // If value not found
          if (value === null) {
            // Console log statement
            console.log(` Variable '${variable}' not found in symbol table.` );
          }
          // Else if the value is found
          else {
            // Console log statement
            console.log(` Variable '${variable}' has value: ${value} `);
          }
        }
      })
  }
}

```

```

        // Show the menu
        menu_hashtable();
    });
    break;
// case 4
case '4':
    // Question for user input to delete variable
    question.question("Delete Variable (A-Z): ", (input) => {
        // Trim any spaces
        const variable = input.trim();
        // If variable is not valid
        if (!variable_valid(variable)) {
            // Console log the error message
            console.log("Error: Only uppercase variable names A-Z are
accepted.");
        }
        // Else if the variable is valid
        else {
            // Search for the value
            const value = variable_search(variable);
            // If value is not found
            if (value === null) {
                // Console log statement
                console.log(` Variable '${variable}' not found in symbol table.` );
            }
            // If the value is found
            else {
                // delete the variable from symbol table
                variable_delete(variable);
                // Console log statement
                console.log(` Variable '${variable}' deleted from symbol table.` );
            }
        }
    }

    // Show the menu
    menu_hashtable();
});
break;
// Case 5
case '5':
    // Back to symbol table menu
    back_menu();
    break;
default:
    // Console log statement for invalid option
    console.log("Invalid option. Please enter 1-5.");
    // Show menu again

```

```

        menu_hashtable();
    }
}

// Allow multiple inputs from the user
function input_mode() {
    // show > as starting the console
    question.question('> ', (line) => {
        // if the user types menu
        if (line.trim().toLowerCase() === 'menu') {
            // Go back to the menu
            menu_hashtable();
            return;
        }
        // Evaluate the postfix expression
        const result = evaluate_postfix(line);
        // Console log the result
        console.log("Result:", result);
        // Ask for the next input
        input_mode();
    });
}

// Start the menu
menu_hashtable();
}

// Export function to use in main.js
module.exports = hash_table;

```

main.js

```

// Main.js file contains the main menu of the program for the user
// Readline package for the user input
const readline = require('readline');

// Input of the Inorder Transversal javascript file
const Inorder_transversal = require('./Symbol_table/Inorder_transveral');

// Input of the Binary Search javascript file
const Binary_search = require('./Symbol_table/Binary_search');

// Input of the Direct Addressing javascript file
const direct_addressing = require('./Symbol_table/direct_addressing');

// Input of the Hash Table javascript file
const hashtable = require('./Symbol_table/hashtable');

```

```

// Input of the arithmetic operations file
const { bubblesort, insertionsort, mergesort, quicksort, selectionsort } =
require('./Arithmetic_operations/Arithmetic');

// Interface for reading input and writing output
const question = readline.createInterface({
  input: process.stdin, // Input
  output: process.stdout // Output
});

// Function for the main menu
function Main_Menu() {
  // Console log statements for the different options for the user to choose
  console.log("\n=====");
  console.log("    Welcome to PostFix Calculator!");
  console.log("=====\\n");
  console.log("1. Arithmetic_operations");
  console.log("2. Symbol_table");
  console.log("3. Exit\\n");
  // Question for the user to enter their input
  question.question("Enter option (1-3): ", Menu_handle);
}

// Different case options for the user input
function Menu_handle(option) {
  // Trim option for the user input in case the user types any white spaces in
  // their input
  switch (option.trim()) {
    // Case 1 shows the arithmetic menu for the user
    case '1':
      Arithmetic_Menu();
      break;
    // Case 2 shows the symbol table menu
    case '2':
      SymbolTable_Menu();
      break;
    // Case 3 exits the program
    case '3':
      console.log("Exiting program. Goodbye!");
      question.close();
      break;
    // Console log statement if the user inputs invalid option and show menu
    // again for the user to choose again
    default:
      console.log("Invalid option. Please enter 1, 2, or 3.");
      Main_Menu();
  }
}

```

```

    }
}

// Menu for Arithmetic Operations
function Arithmetic_Menu() {
    // Console log statements for arithmetic operations menu
    console.log("\n=====");
    console.log("    Arithmetic Operations Menu    ");
    console.log("=====\\n");
    console.log("1. Bubble Sort");
    console.log("2. Insertion Sort");
    console.log("3. Merge Sort");
    console.log("4. Quick Sort");
    console.log("5. Selection Sort");
    console.log("6. Back to Main Menu\\n");
    // Question for the user to input option 1 to 6
    question.question("Choose an option (1-6): ", Arithmetic_handle);
}

// Different case options for the user input
function Arithmetic_handle(option) {
    // Trim option for the user input incase the user types any white spaces in
    // their input
    switch (option.trim()) {
        // Case 1 goes to the bubble sort menu options
        case '1':
            bubblesort(question, Arithmetic_Menu);
            return;
        // Case 2 goes to the insertion sort menu options
        case '2':
            insertionsort(question, Arithmetic_Menu);
            return;
        // Case 3 goes to the merge sort menu options
        case '3':
            mergesort(question, Arithmetic_Menu);
            return;
        // Case 4 goes to the quick sort menu options
        case '4':
            quicksort(question, Arithmetic_Menu);
            return;
        // Case 5 goes to the selection sort menu options
        case '5':
            selectionsort(question, Arithmetic_Menu);
            return;
        // Case 6 goes back to the main menu options
        case '6':
            Main_Menu();
    }
}

```

```

        return;
        // Console log statement if the user input invalid options
        default:
            console.log("Invalid option. Please choose between 1 and 6.");
    }
    // Loop back to the arithmetic menu
    Arithmetic_Menu();
}

// Menu for Symbol Table
function SymbolTable_Menu() {
    // Console log statements for the symbol table menu
    console.log("\n=====");
    console.log("        Symbol Table Menu        ");
    console.log("=====\\n");
    console.log("1. Inorder Transversal");
    console.log("2. Binary Search");
    console.log("3. Direct Addressing");
    console.log("4. Hash Table");
    console.log("5. Back to Main Menu\\n");
    // Question for the user to input option 1 to 5
    question.question("Choose an option (1-5): ", SymbolTable_handle);
}

// Different case options for the user input
function SymbolTable_handle(option) {
    // Trim option for the user input incase the user types any white spaces in
    their input
    switch (option.trim()) {
        // Case 1 goes to the inorder transversal menu options
        case '1':
            Inorder_transversal(question, SymbolTable_Menu);
            return;
        // Case 2 goes to the binary search menu options
        case '2':
            Binary_search(question, SymbolTable_Menu);
            return;
        // Case 3 goes to the direct addressing menu options
        case '3':
            direct_addressing(question, SymbolTable_Menu);
            return;
        // Case 4 goes to the hashtable menu options
        case '4':
            hashtable(question, SymbolTable_Menu);
            return;
        // Case 5 go back to the main menu options
        case '5':

```



```

    Main_Menu();
    return;
    // Console log statement if the user input invalid options between 1 to 5
    default:
        console.log("Invalid option. Please choose between 1 and 5.");
    }
    // Loop back to the symbol table menu
    SymbolTable_Menu();
}

// Start the application
Main_Menu();

```

6. Defects and Remedies

Weaknesses of your solution

Hash Table Collisions

Using a hash table to store variable values is one of my solution's main weaknesses. Even though hash tables are quick and effective, they can encounter a common issue known as a collision, in which two distinct variable names end up in the same storage location. For example, if variables A and Z are stored using the same hash function and produce the same result, one may overwrite the other or result in lookup errors. Collisions are not handled in my current implementation, which could result in inaccurate results or data loss.

I could use the common methods covered in this module to correct this. Open Addressing, which is the program uses either linear or quadratic probing to find the next available slot if a slot is already taken. Rehashing, which is increase the table size and use a better hash function to move all the variables that are currently there to new locations if there are too many collisions. I could greatly increase the reliability and robustness of my hash table for all variable storage and lookup requirements by including one of these solutions.

Direct Addressing Limitations

Although it is very effective, direct addressing, which I used as a backup strategy for storing variables A–Z, has disadvantages of its own. The only reason it works is that there are only a few known variables exactly 26. In the future, this structure would not function if the Postfix++ language were extended to allow multi-letter variable names such as AA, ABC, or even lowercase variables. This could make the fixed array unusable or inefficient. A dynamic mapping structure, such as a binary search tree that supports scalable, sorted, and dynamic keys or

a hash table with appropriate collision handling, would have to be used in place of the direct addressing table to fix this.

Operator Limitations

The calculator only supports the four fundamental operators of addition (+), subtraction (-), multiplication (*), and division (/), which is another weakness in my solution. Any other operator, such as modulus (%), exponentiation (^), or logical operations, is regarded as invalid. The program only prints an error message if a user accidentally enters such a symbol. It is unable to evaluate the expression further. This restricts the calculator's flexibility. I could add custom logic for operators inside the evaluation function to expand the expression evaluator's recognition capabilities. For example, I could use JavaScript's `Math.pow(a, b)` and check for the ^ symbol to implement exponentiation. Additionally, `a % b` could be used to add the modulus operator. For the interpreter to correctly recognize these symbols, I would also update the symbol validation rules to include them.

Stack size limitation

The calculator temporarily stores numbers in a stack for all postfix evaluation functions. The stack still has a practical limit based on system memory, even though JavaScript arrays can expand in size dynamically. A lengthy expression entered by the user could cause the stack to grow too big, which could impair performance or even cause the program to crash. I can correct this by adding a check to the program that would stop more than a specific number of values for example, 100 from being pushed onto the stack. The application may stop processing and give a clear message stating "Expression too long" if the limit is reached. By doing this, memory overflow is prevented, and stability is preserved.

Lack of input validation

The limited ability to handle invalid input is another weakness in the design. For example, the calculator shows an error but does not provide the user with an opportunity to fix the input. This may result in a less user-friendly program. I might also give users the option to re-enter their expression using edit options, as well as more informative error messages. These characteristics would strengthen the system's resilience and increase its usability.

I can improve the efficiency, scalability, and usability of my Postfix++ calculator while adhering to algorithm and data structure design guidelines by recognizing these weaknesses and implementing the necessary fixes, such as rehashing, expanding parsing logic, and more thorough input validation.

